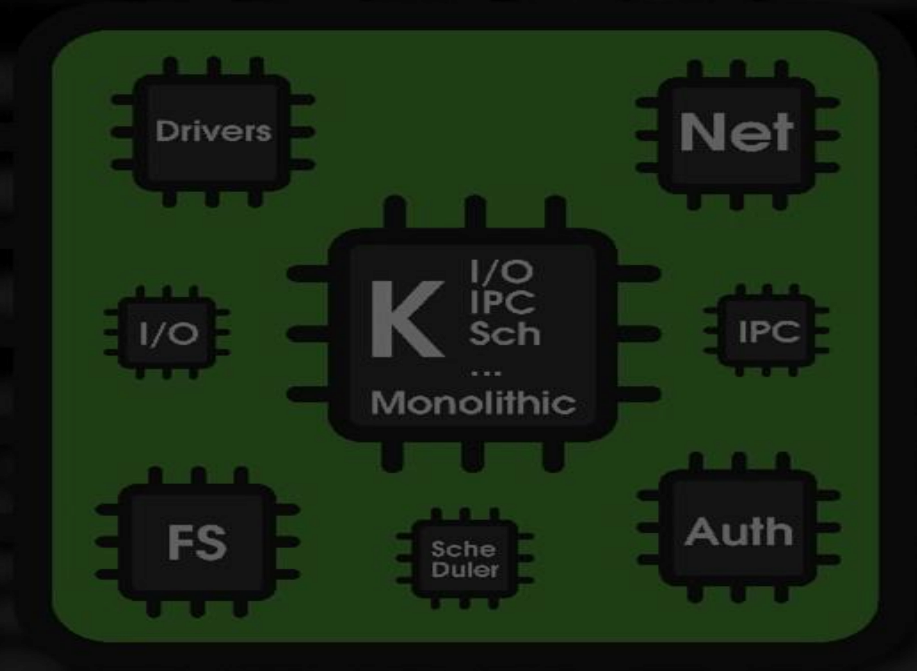


Arquitectura de los Sistemas Operativos



COMPONENTES DEL SO

¿Qué cosas hace un sistema operativo?

Todo SO necesita y siempre existen (funciones de un SO):

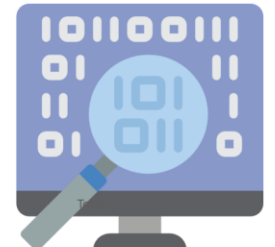
Administración de Procesos: Gestiona la creación, ejecución y finalización de programas (procesos e hilos), además de asignar el tiempo de CPU mediante el Planificador (Scheduler).

Administración de Memoria: Controla el uso de la RAM, asignando espacios seguros para cada proceso y gestionando la Memoria Virtual (Paginación).

Sistema de Archivos (Filesystem): Proporciona una abstracción para organizar, almacenar y recuperar datos en dispositivos de almacenamiento (HDD/SSD).



PROGRAMA



PROCESO

COMPONENTES DEL SO

Controladores de Dispositivos (Drivers): Software especializado que permite al Kernel comunicarse con el hardware específico (impresoras, GPU, teclados).

Subsistema de Red: Gestiona la comunicación entre computadoras, controlando protocolos, envío y recepción de paquetes de datos.

Seguridad y Protección: Controla el acceso a los recursos del sistema mediante permisos, autenticación y políticas de aislamiento entre usuarios.

Interfaz de Usuario (Shell): El intérprete de comandos que permite al usuario interactuar con el sistema operativo y ejecutar otros programas.



COMPONENTES DE UN SISTEMA OPERATIVO: VISTA GENERAL

VISTA INTRODUCTORIA SIN ARQUITECTURAS ESPECÍFICAS



Sistemas Operativos-Llamada al sistema

- Según IBM, una **llamada al sistema** es :

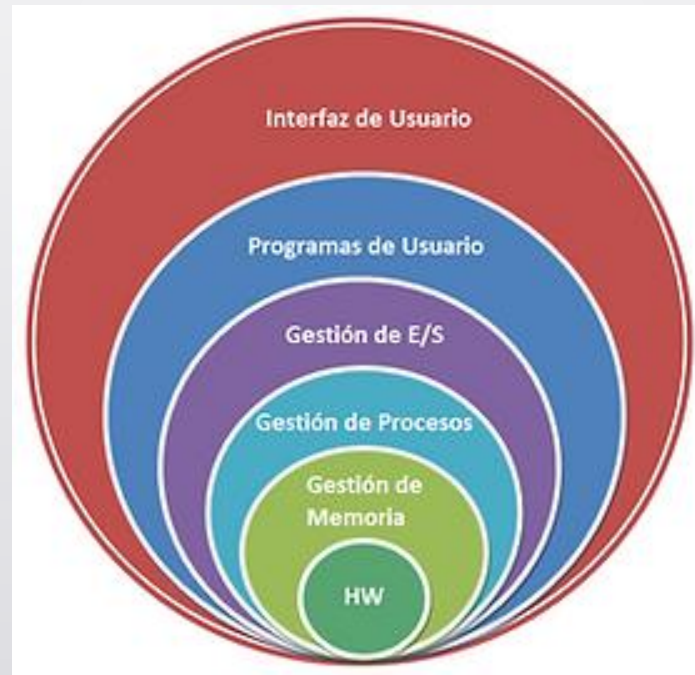
“Una rutina que permite a una aplicación de usuario solicitar acciones que requieren privilegios especiales”

- También es posible extender las funciones proporcionadas por el *kernel*.
- Se accede a través de bibliotecas de programación.



Arquitectura de un Sistema Operativo

La arquitectura de un Sistema Operativo es el modelo de organización de sus componentes definiendo qué partes se ejecutan en Modo Kernel, qué partes se ejecutan en Modo Usuario, cómo se comunican entre sí y cómo se separan las responsabilidades.



Arquitectura de un Sistema Operativo

¿Cómo está organizado internamente un Sistema Operativo?

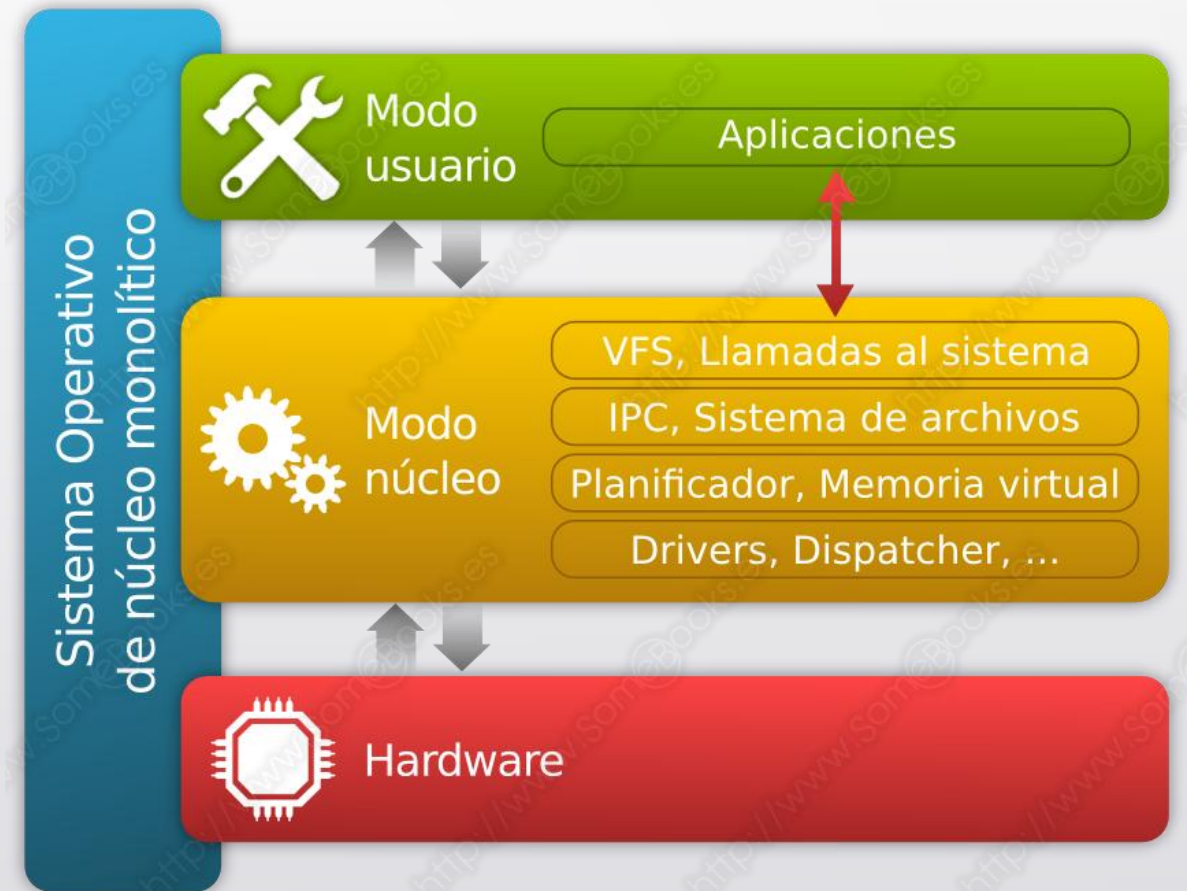
Todas las arquitecturas organizan los mismos componentes.

- Monolítico
- Microkernel
- Híbrido
- Exokernel

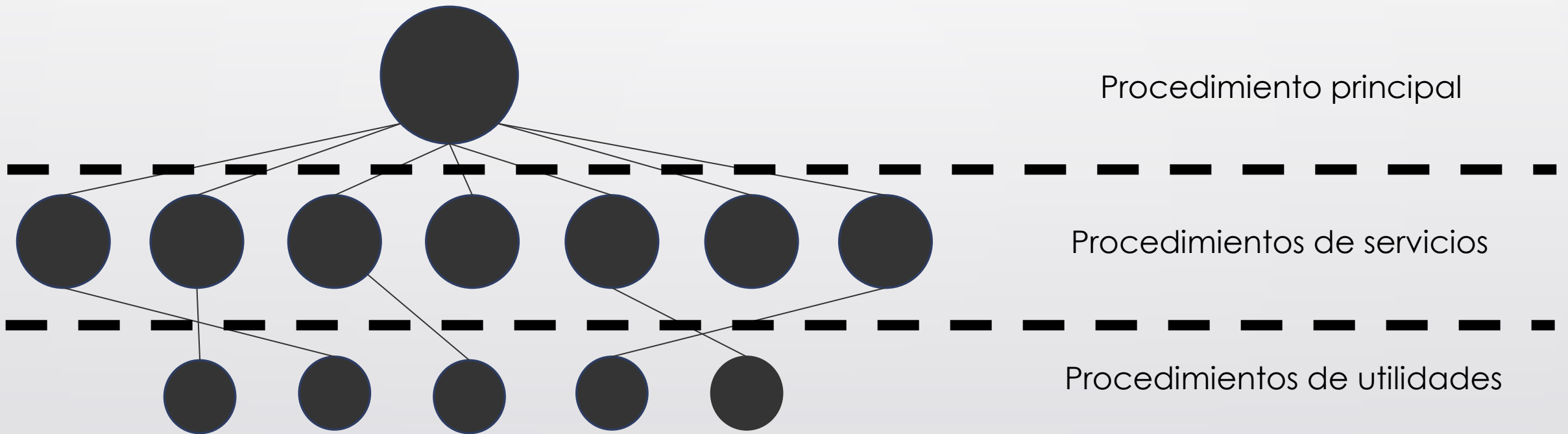


Arquitectura Monolítica

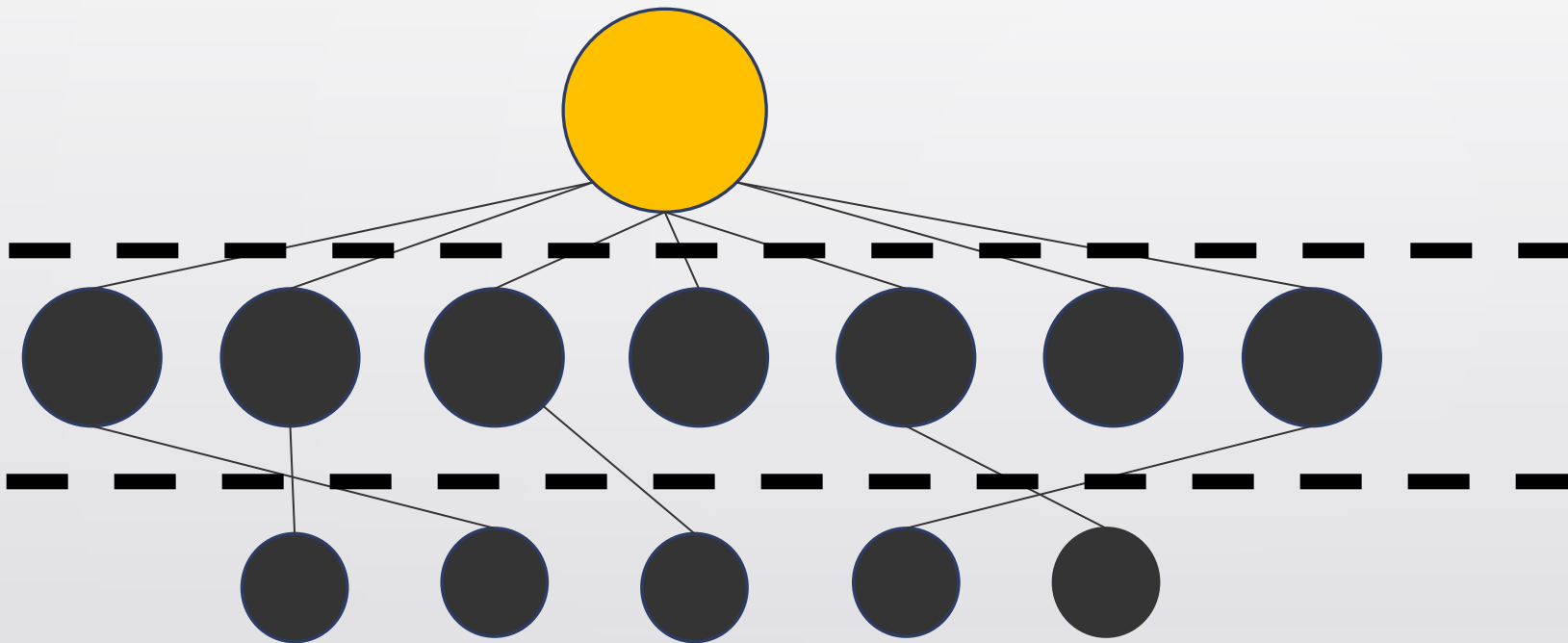
- Es una arquitectura donde el núcleo del sistema operativo es un bloque grande y unificado.
- Incluye todos los servicios esenciales (gestión de procesos, memoria, sistema de archivos, controladores de dispositivos, red, etc.)
- Se ejecutan juntos en el mismo espacio de memoria y con máximo privilegio.
- Ejemplo ¿?



Sistemas monolíticos



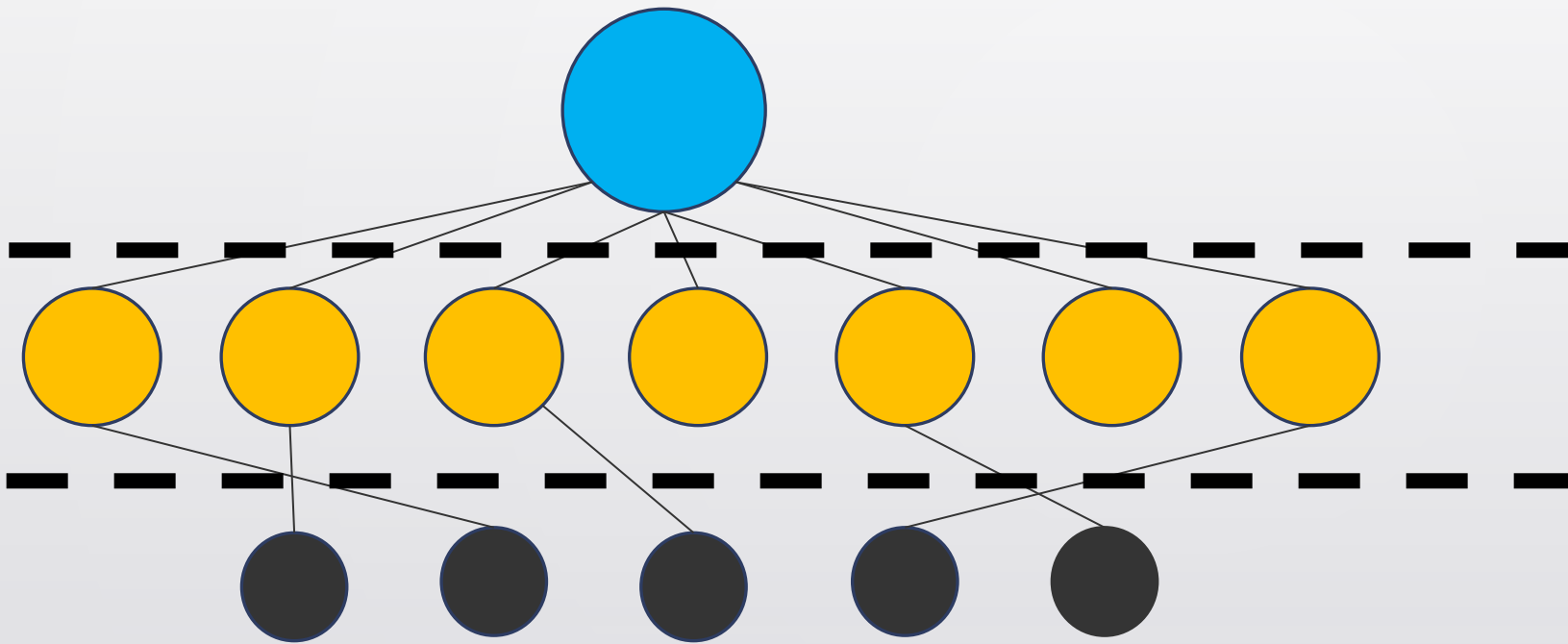
Sistemas monolíticos



Procedimiento principal

- Invoca a los procedimientos de servicio

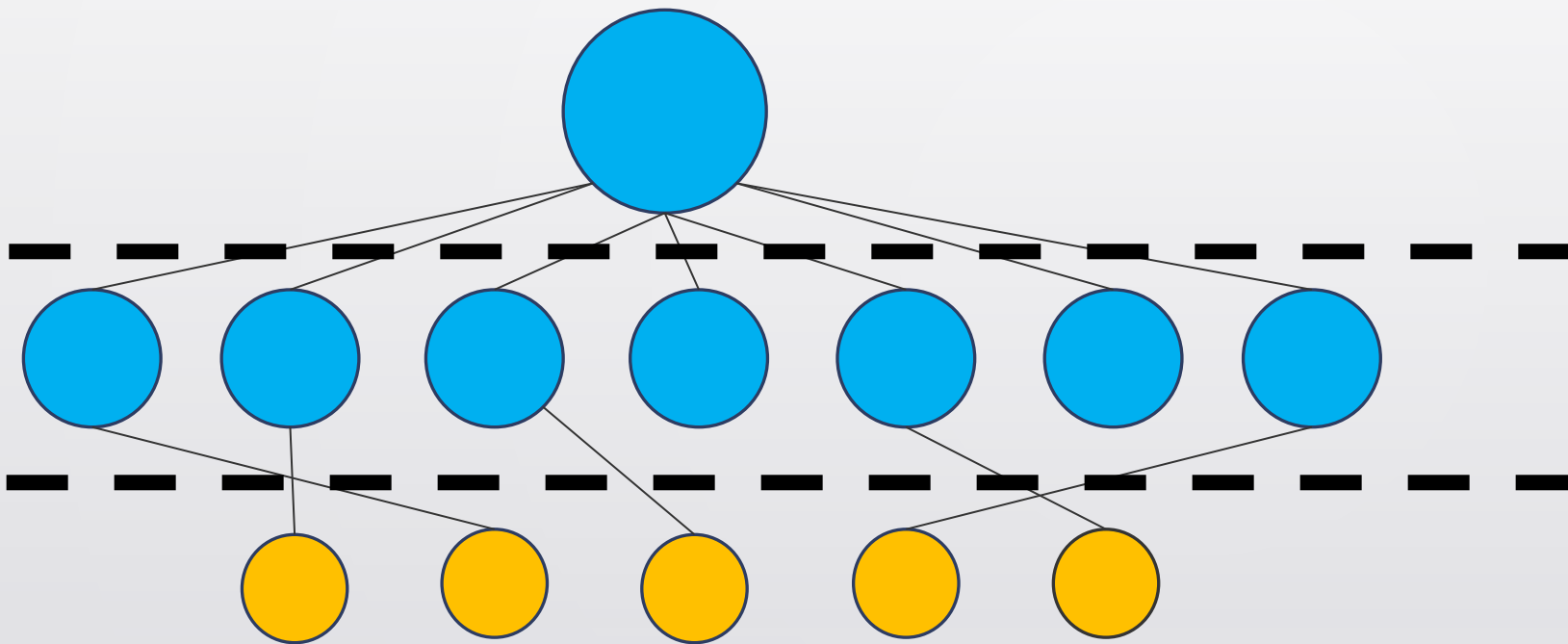
Sistemas monolíticos



Procedimientos de servicio

- Hay un procedimiento por cada llamada al sistema

Sistemas monolíticos



Procedimientos de utilidades

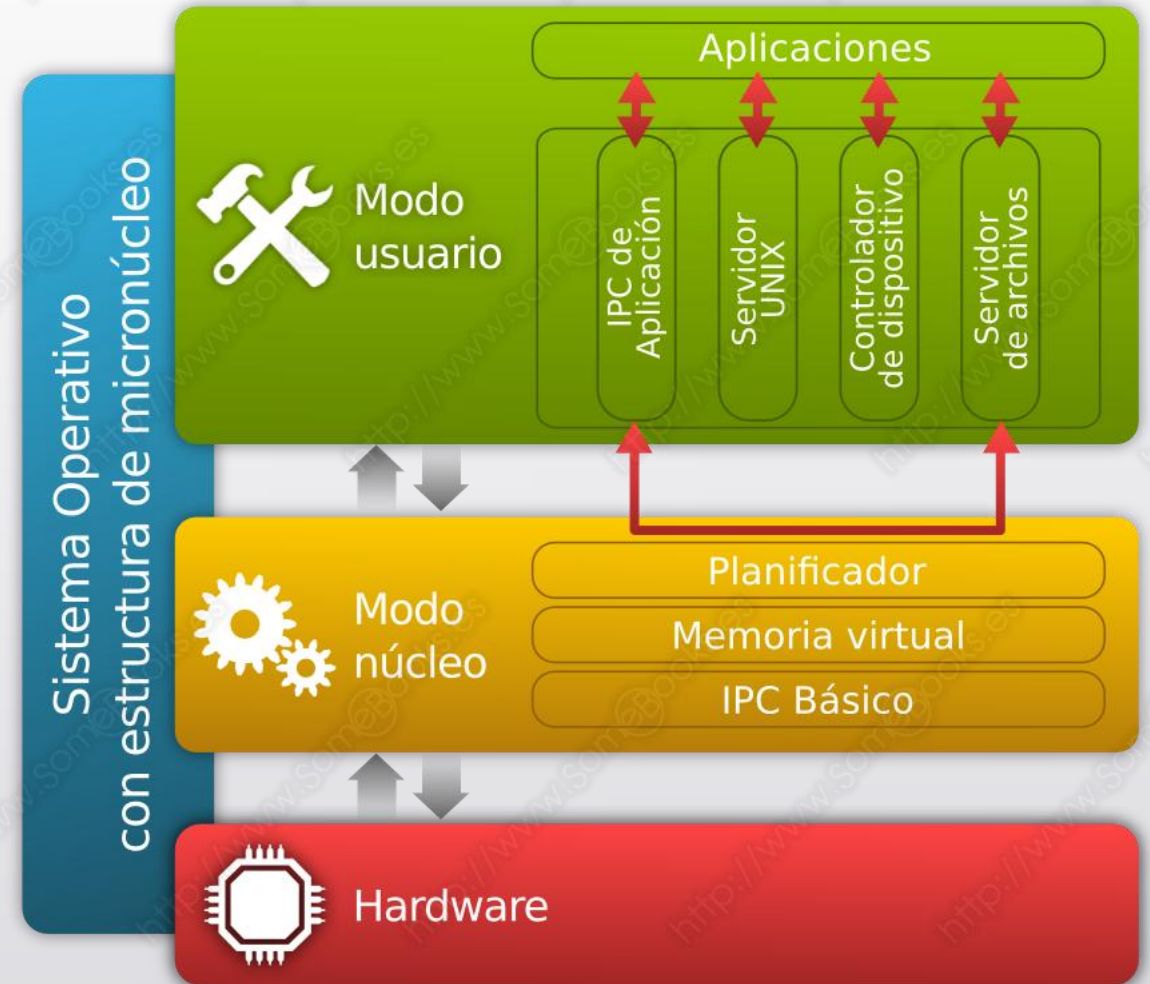
- Realizan funciones necesarias para los procedimientos de servicios, es decir, los ayudan a realizar sus tareas.

Arquitectura Monolítica ventajas y desventajas

Característica	Ventajas	Desventajas
Rendimiento	Máxima velocidad: Comunicación directa entre componentes (llamadas a funciones) sin cambio de modo.	Complejidad de gestión: El código se vuelve difícil de mantener a medida que crece el sistema.
Estructura	Diseño simple inicialmente: Menos complejo de implementar al no requerir paso de mensajes (IPC).	Falta de modularidad real: Los componentes están fuertemente acoplados.
Estabilidad	Eficiencia de recursos: Menor sobrecarga de memoria y CPU al evitar cambios de contexto constantes.	Baja tolerancia a fallos: Un error en un driver puede provocar un colapso total (Kernel Panic).
Seguridad	Control centralizado: Gestión unificada de todos los recursos del sistema.	Riesgo de seguridad: Un driver comprometido tiene acceso total a todo el sistema.
Mantenimiento	Menos latencia: Ideal para sistemas de tiempo real o alto rendimiento.	Dificultad para depurar: Es difícil aislar errores cuando todo corre en el mismo espacio.

Arquitectura Microkernel

- Kernel muy pequeño, solo contiene lo indispensable.
- Incluye solo los servicios esenciales (Gestión de interrupciones, mecanismos de comunicación (IPC), planificador de CPU (Scheduler), gestión básica de memoria.
- Todos los demás componentes se ejecutan en modo usuario.
- Se ejecutan de forma aislada en espacios de memoria separados y con privilegios restringidos.
- Ejemplo ¿?



Arquitectura Microkernel ventajas y desventajas

Característica	Ventajas	Desventajas
Fiabilidad	Tolerancia a fallos: Si un driver o servicio falla, se puede reiniciar sin que el sistema colapse.	Sobrecarga de comunicación: El paso de mensajes (IPC) genera una latencia mayor que las llamadas directas.
Seguridad	Aislamiento total: Los componentes corren con privilegios mínimos en Modo Usuario.	Complejidad de diseño: Implementar un sistema de mensajería seguro y eficiente entre procesos es difícil.
Mantenimiento	Modularidad extremo: Es muy fácil añadir o probar nuevos servicios sin recompilar el núcleo.	Pérdida de rendimiento: Los constantes cambios de contexto entre Modo Usuario y Modo Kernel consumen CPU.
Tamaño	Núcleo compacto: El código del kernel es pequeño, lo que facilita su verificación y depuración.	Gestión de memoria compleja: Mover datos entre espacios de usuario requiere una administración avanzada.
Rendimiento	Adaptabilidad: Ideal para sistemas distribuidos y embebidos donde la flexibilidad es clave.	Dificultad de optimización: Es más difícil lograr la misma velocidad que un sistema monolítico.

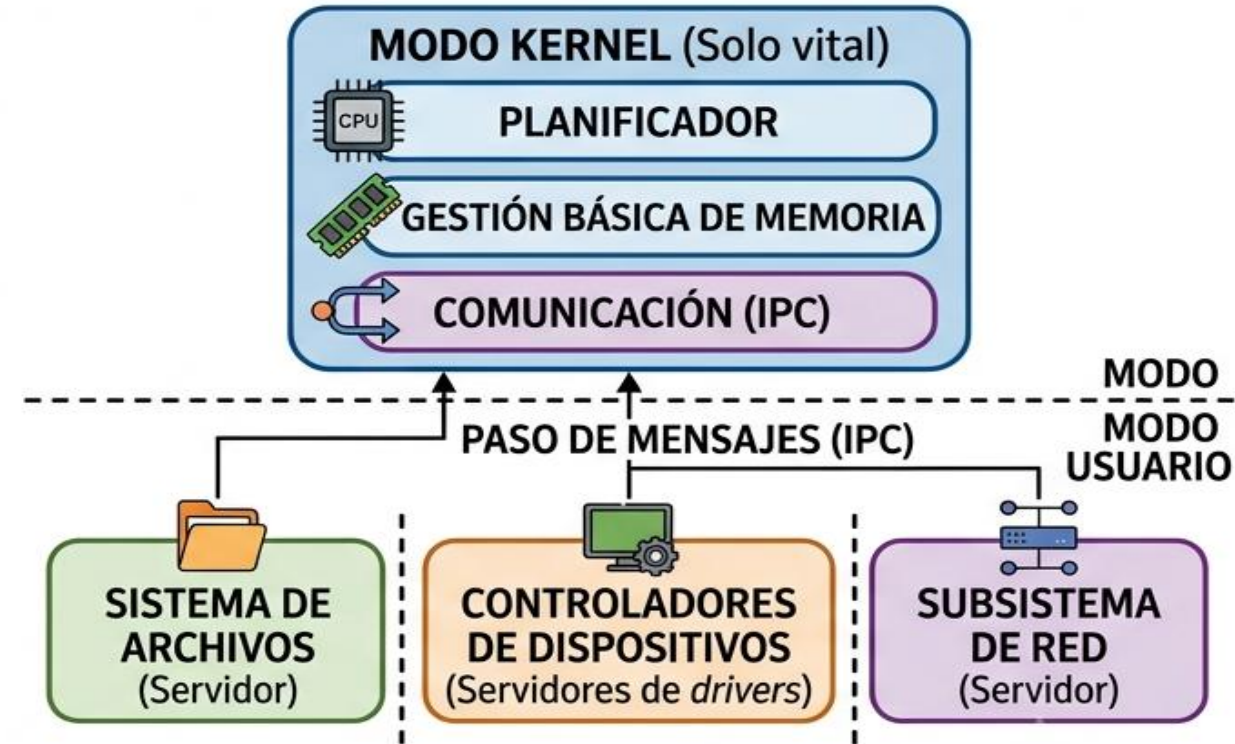
COMPONENTES DEL SISTEMA OPERATIVO Y ARQUITECTURAS

ARQUITECTURA MONOLÍTICA



Ventajas y Desventajas:
Monolítico: Rápido, frágil si falla driver

ARQUITECTURA MICROKERNEL



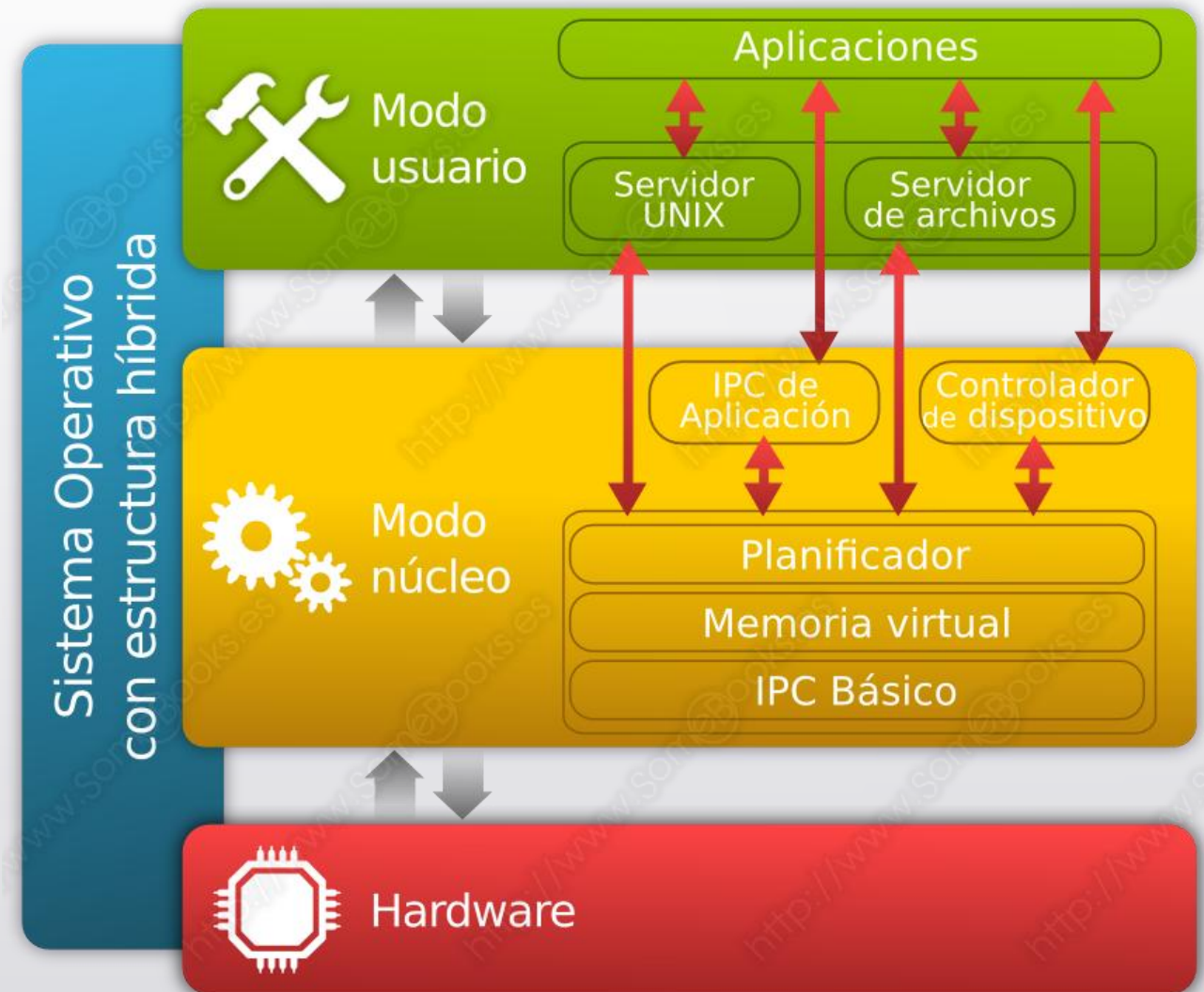
Ventajas y Desventajas:
Microkernel: Muy seguro (fallo de driver no mata kernel), más lento por IPC



¿Qué será una arquitectura híbrida?

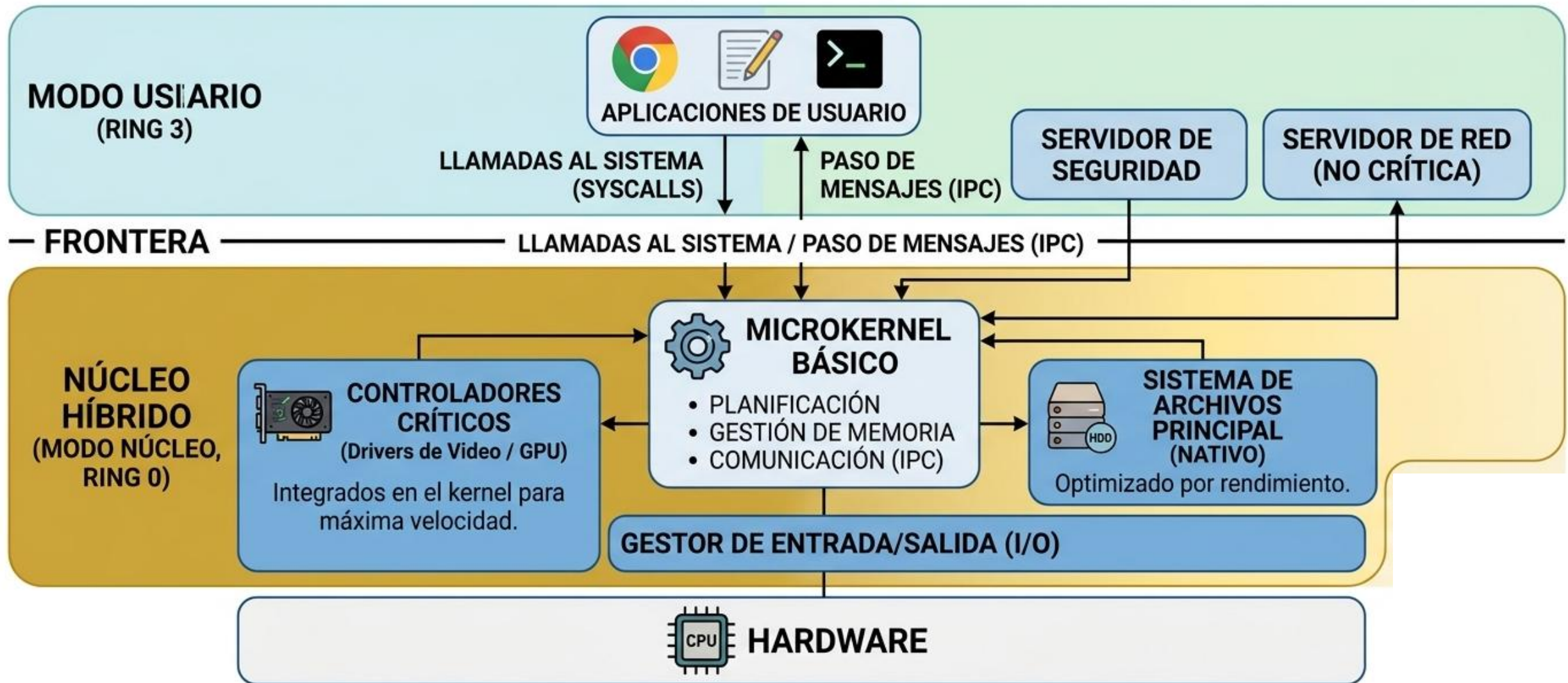
Arquitectura Híbrida

- Consiste en un esquema de microkernel que incluye algo de código complementario para hacerlo más rápido.
- Aunque buena parte de las funciones del sistema operativo siguen ejecutándose en modo usuario.
- Componentes en el modo kernel: Gestión de memoria, Scheduling (planificación), IPC (comunicación entre Procesos) optimizado, algunos drivers críticos (disco, red básica).
- Componentes en modo usuario: la mayoría de los drivers de dispositivos, sistemas de archivos, servicios de red avanzados, subsistemas gráficos.
- La han adoptado muchos de los sistemas operativos modernos.
- Ejemplo ¿?



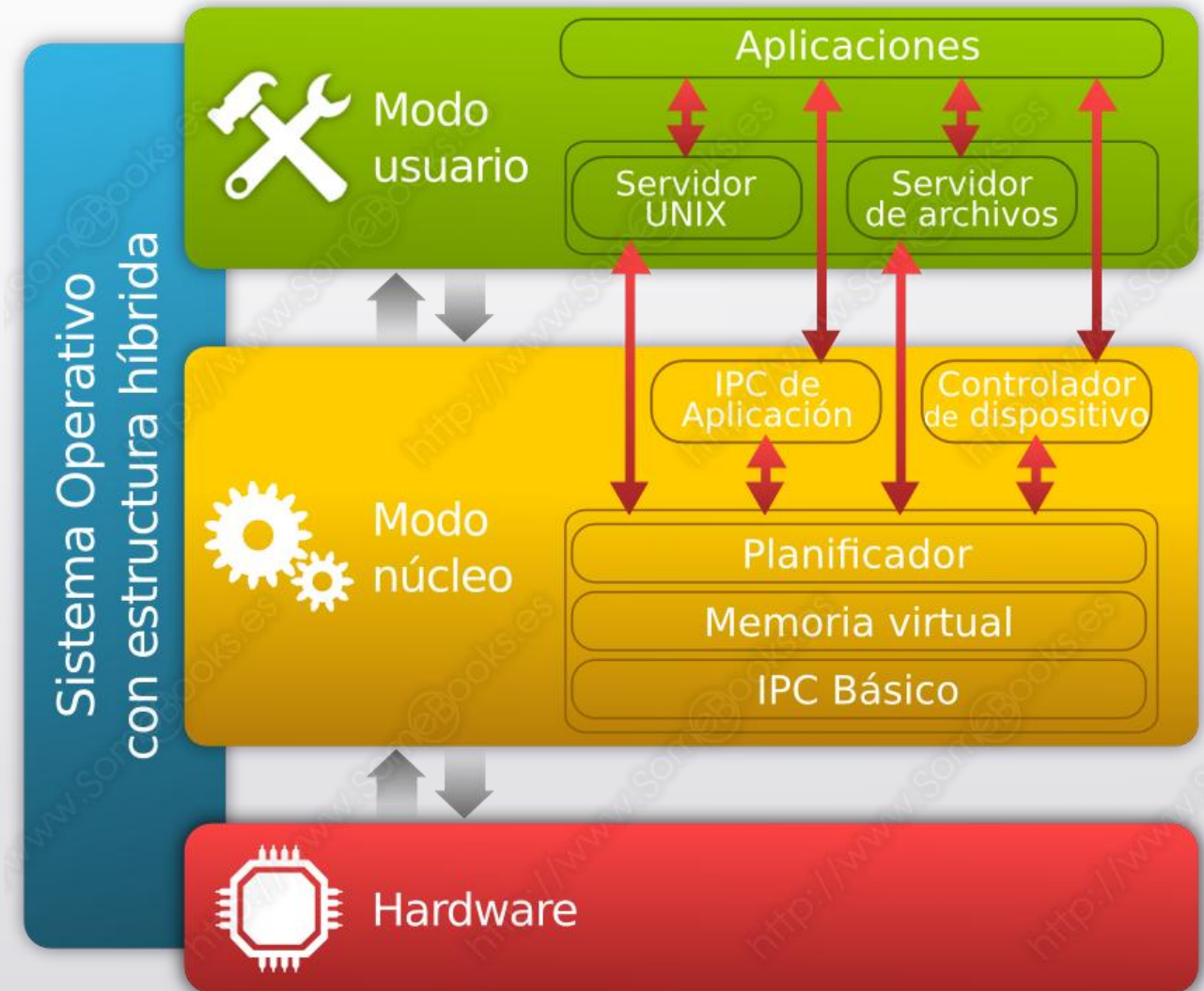
ARQUITECTURA HÍBRIDA: VISTA CONCEPTUAL

EQUILIBRIO ENTRE RENDIMIENTO (MONOLÍTICO) Y MODULARIDAD (MICROKERNEL)



Arquitectura Exokernel

- Diseño de kernel más radicales y minimalistas que existen.
- Propuesto por investigadores del MIT en la década de 1990.
- Exokernel solo se encarga : **Proteger los recursos** (evitar que un programa dañe a otro), **multiplexar** (compartir) los recursos de forma segura, autorizar los accesos.
- Todo lo demás (políticas de scheduling, manejo de memoria virtual, sistemas de archivos, etc.) se deja a bibliotecas en espacio de usuario.
- Cada aplicación puede tener su propia "mini-sistema operativo" personalizado.
- Ejemplo ¿?



Análisis del modelo Exokernel

✓ Ventajas

- ⚡ **Máximo rendimiento:** Las aplicaciones acceden casi directamente al hardware sin capas innecesarias.
- ⚙️ **Personalización extrema:** Cada programa puede implementar su propio scheduler o sistema de archivos.
- 🎯 **Eficiencia:** Se elimina el "overhead" de las abstracciones forzadas por el kernel tradicional.
- 💡 **Innovación:** Facilita la experimentación con nuevas técnicas de gestión de recursos.

✗ Desventajas

- </> **Complejidad:** Los desarrolladores deben manejar detalles de bajísimo nivel, aumentando la carga de trabajo.
- 🔧 **Poco portable:** Las aplicaciones se vuelven altamente dependientes del hardware específico.
- 🔍 **Depuración difícil:** Los errores en las librerías de sistema pueden ser críticos y difíciles de aislar.
- 🎓 **Baja adopción:** Es principalmente un concepto académico y de investigación especializada.

Comparativa de Arquitecturas de SO

ASPECTO	MONOLÍTICO	MICROKERNEL	HÍBRIDO	EXOKERNEL
Tamaño del kernel	Muy grande	Pequeño	Mediano	Mínimo
Abstracción	Alto	Medio	Medio	Muy bajo (Raw)
Rendimiento	Alto	Medio-Bajo	Alto	Máximo potencial
Flexibilidad	Baja	Alta	Alta	Extremadamente alta
Complejidad Devs	Media	Alta	Media	Muy alta
Seguridad	Media	Alta	Alta	Alta (Aislado)
Uso actual	Muy común	Nicho	Muy común	Investigación